What is the difference between pip and conda?

Asked 9 years, 10 months ago Modified 5 months ago Viewed 527k times

1082

I know `pip` is a package manager for python packages. However, I saw the installation on IPython's website use `conda` to install IPython.

Can I use `pip` to install IPython? Why should I use `conda` as another python package manager when I already have `pip`?

What is the difference between `pip` and `conda`?

python pip ipython package-managers conda

Share Improve this question Follow

edited Jul 25, 2017 at 15:36
serv-inc
36.1k 9 169 190

asked Jan 8, 2014 at 11:44
lazywei
12k 5 20 25

17 One difference is that many more things can be installed by pip than by conda: pip can install anything from pypi in one command. conda requires three commands: skeleton, build, install and possibly more if that doesn't work. pip can install anything from github or source in one command. conda requires writing a "recipe", which is not easy, especially since the documentation always seems to be incorrect/outdated. – endolith Feb 22, 2016 at 5:31

11 Related question: What are the ADVANTAGES of pip over conda? I see lots of Anaconda advocacy below, but nothing for pip. Why is pip still the standard, if anaconda is so great? – Brian Postow Mar 16, 2016 at 21:20

73 I find this quote enlightening: "*Pip* is a package manager, and *Virtualenv* is an environment manager. *Conda* is both." (ref.) – Atcold Jan 20, 2017 at 16:29

7 Observation: I used to think conda implied downloading a zillion packages, but this is apparenlty no longer true: you can install miniconda, which seems to essentially be just the package manager, conda.io/docs/install/quick.html – Hugh Perkins Jun 29, 2017 at 10:03

2 @RichLysakowskiPhD How do you install from pypi? – endolith Sep 11, 2020 at 4:18

14 Answers

Sorted by: Highest score (default)

721

Quoting from the Conda blog:

> Having been involved in the python world for so long, we are all aware of pip, easy_install, and virtualenv, but these tools did not meet all of our specific requirements. The main problem is that they are focused around Python, neglecting non-Python library dependencies, such as HDF5, MKL, LLVM, etc., which do not have a setup.py in their source code and also do not install files into Python's site-packages directory.

So Conda is a packaging tool and installer that aims to do more than what `pip` does; handle library dependencies *outside* of the Python packages as well as the Python packages themselves. Conda also creates a virtual environment, like `virtualenv` does.

As such, Conda should be compared to Buildout perhaps, another tool that lets you handle both Python and non-Python installation tasks.

Because Conda introduces a new packaging format, you cannot use `pip` and Conda interchangeably; `pip` cannot install the Conda package format. You can use the two tools side by side (by installing `pip` with `conda install pip`) but they do not interoperate either.

Since writing this answer, Anaconda has published a new page on *Understanding Conda and Pip*, which echoes this as well:

> This highlights a key difference between conda and pip. Pip installs Python packages whereas conda installs packages which may contain software written in any language. For example, before using pip, a Python interpreter must be installed via a system package manager or by downloading and running an installer. Conda on the other hand can install Python packages as well as the Python interpreter directly.

and further on

> Occasionally a package is needed which is not available as a conda package but is available on PyPI and can be installed with pip. In these cases, it makes sense to try to use both conda and pip.

Share Improve this answer Follow

edited Aug 27, 2021 at 14:03

answered Jan 8, 2014 at 11:47

Martijn Pieters ♦
1.1m 303 4080 3362

26 Thanks for your explanation. I'm still confused, however, with whether can Conda replace pip? i.e., can Conda do all what pip can do? – lazywei Jan 8, 2014 at 12:03

11 @lazywei: I don't think it can; it doesn't look like Conda supports the wheel archive format, for example. They have different aims. – Martijn Pieters ♦ Jan 8, 2014 at 12:04

61 @naught101 "Just create a recipe" That's not as easy as typing `pip install .` – endolith Feb 20, 2016 at 22:40

31 Can some explain to me what would happen if you used pip and conda to install the same package except with different version, which would python use when you import them? – Lance Ruo Zhang Jan 19, 2017 at 8:23

4 @jrh: you can't install multiple versions of the same package side by side when using the OS package manager, not as a core feature. Conda manages *environments*, each with their own mix of installed packages at specific versions. So project A, having been developed some time ago, can still cling on to an older version of library Foo (no resources available to upgrade the project to work with a newer release) while project B requires the newest version of the same library. – Martijn Pieters ♦ Nov 12, 2018 at 17:06

300

Disclaimer: This answer describes the state of things as it was a decade ago, at that time pip did not support binary packages. Conda was specifically created to better support building and distributing binary packages, in particular data science libraries with C extensions. For reference, pip only gained widespread support for portable binary packages with wheels (pip 1.4 in 2013) and the manylinux1 specification (pip 8.1 in March 2016). See the more recent answer for more history.

Here is a short rundown:

pip

- Python packages only.
- Compiles everything from source. **EDIT: pip now installs binary wheels, if they are available.**
- Blessed by the core Python community (i.e., Python 3.4+ includes code that automatically bootstraps pip).

conda

- Python agnostic. The main focus of existing packages are for Python, and indeed Conda itself is written in Python, but you can also have Conda packages for C libraries, or R packages, or really anything.
- Installs binaries. There is a tool called `conda build` that builds packages from source, but `conda install` itself installs things from already built Conda packages.
- External. conda is an environment and package manager. It is included in the Anaconda Python distribution provided by Continuum Analytics (now called Anaconda, Inc.).

conda is an environment manager written in Python and is language-agnostic. conda environment management functions cover the functionality provided by venv, virtualenv, pipenv, pyenv, and other Python-specific package managers. You could use conda within an existing Python installation by pip installing it (though this is not recommended unless you have a good reason to use an existing installation). As of 2022, conda and pip are not fully aware of one another package management activities within a virtual environment, not are they interoperable for Python package management.

In both cases:

- Written in Python
- Open source (conda is BSD and pip is MIT)
- Warning: While conda itself is open-source, the package repositories are hosted by Anaconda Inc and have restrictions around commercial usage.

The first two bullet points of conda are really what make it advantageous over pip for many packages. Since pip installs from source, it can be painful to install things with it if you are unable to compile the source code (this is especially true on Windows, but it can even be true on Linux if the packages have some difficult C or FORTRAN library dependencies). conda installs from binary, meaning that someone (e.g., Continuum) has already done the hard work of compiling the package, and so the installation is easy.

There are also some differences if you are interested in building your own packages. For instance, pip is built on top of setuptools, whereas conda uses its own format, which has some advantages (like being static, and again, Python agnostic).

Share Improve this answer Follow

edited Dec 11, 2022 at 14:34
user5994461
5,607 3 36 58

answered Jan 8, 2014 at 23:37
asmeurer
87.4k 26 170 240

24 pip no longer builds everything from source. If a wheel is available, `pip install --use-wheel <package>` will install a built package. See here: wheel.readthedocs.org/en/latest. However my personal experience with wheel is that so few scientific wheel packages are available that it is purely academic. And of course pip install mostly doesn't work either on windows if your build environment isn't set up exactly right. So at the moment, conda ftw. – Caleb Hattingh Jan 9, 2014 at 0:37

4 Wheels are still new, and not used by default, so it's not surprising that there aren't really many of them yet. Wheel still fits into the category of "Python specific", though, meaning it can be a poor fit for non-Python packages, or Python packages that depend on non-Python packages. – asmeurer Jan 11, 2014 at 20:31

5 I had to downvote this: the second bullet point is just a historical note now, but you go on it later on too. The main difference these days is that pip is a package manager while conda is more of an environment manager. – Shep Dec 22, 2016 at 15:44

12 It's true that pip can compile from source but this is becoming less and less frequent as more package move to wheel: these days I can install most of what I need in a few seconds with pip. So it's not that this answer is wrong, it's just becoming slightly outdated as pip has improved quite a lot in the last few years – Shep Dec 24, 2016 at 19:21

9 It has ***never*** been true that pip can only install from source. Before wheels we had eggs as the default binary install format and the recommended distribution format for Windows installations, and pip would (and still will) install eggs if that's the best available option. – Martijn Pieters ♦ Mar 3, 2019 at 9:59

116

The other answers give a fair description of the details, but I want to highlight some high-level points.

pip is a package manager that facilitates installation, upgrade, and uninstallation of **python packages**. It also works with virtual **python** environments.

conda is a package manager for **any software** (installation, upgrade and uninstallation). It also works with virtual **system** environments.

One of the goals with the design of conda is to facilitate package management for the entire software stack required by users, of which one or more python versions may only be a small part. This includes low-level libraries, such as linear algebra, compilers, such as mingw on Windows, editors, version control tools like Hg and Git, or *whatever else requires distribution and management*.

For version management, pip allows you to switch between and manage multiple **python** environments.

Conda allows you to switch between and manage **multiple general purpose environments** across which multiple other things can vary in version number, like C-libraries, or compilers, or test-suites, or database engines and so on.

Conda is not Windows-centric, but on Windows it is by far the superior solution currently available when complex scientific packages requiring compilation are required to be installed and managed.

I want to weep when I think of how much time I have lost trying to compile many of these packages via pip on Windows, or debug failed `pip install` sessions when compilation was required.

As a final point, Continuum Analytics also hosts (free) binstar.org (now called anaconda.org) to allow regular package developers to create their own custom (built!) software stacks that their package-users will be able to `conda install` from.

Share Improve this answer Follow

edited Feb 11, 2019 at 4:55
kalefranz
4,642 2 27 42

answered Jan 9, 2014 at 1:13
Caleb Hattingh
9,035 2 32 46

7 Per your final point, the third-party **conda-forge** project has rapidly become the industry-standard approach to publishing Anaconda packages. We recently published several conda-forge-hosted packages for our multiphysics biology simulator – and cannot recommend the process enough. There's a GitHub PR-based peer review component to submitting new recipes to conda-forge, but the advantages in terms of conda-forge automation strongly outweigh the upfront time investment. *Bam!* – Cecil Curry Jun 7, 2018 at 3:59

@CecilCurry I've imported `Keras` in my code, installed anaconda on my mac and Keras is both `conda` installed and `pip` installed. So, when running my code in terminal, how do I know which `keras` is being imported(the `pip` one or the `conda` one)? – asn Dec 31, 2018 at 12:48

@Caleb Hattingh this is a great answer! Getting the things compiled on Windows requires a deep knowledge Visual Studio. Using conda removes almost all of that (often extreme Microsoft-specific) technical expertise to get the scientific libraries compiled without much effort. conda is a god-send for that. BTW, Anaconda is mostly out of the picture when it comes to conda, except as one of the maintainers, and a big contributor to the conda open source package. – Rich Lysakowski PhD Aug 5, 2022 at 4:39

51

(2021 UPDATE)

TL;DR Use pip, it's the official package manager since Python 3.

pip

- **basics**
- pip is the default package manager for python
- pip is built-in as of Python 3.0
- Usage: `python3 -m venv myenv; source myenv/bin/activate; python3 -m pip install requests`
- Packages are downloaded from pypi.org, the official public python repository
- It can install precompiled binaries (wheels) when available, or source (tar/zip archive).
- Compiled binaries are important because many packages are mixed Python/C/other with third-party dependencies and complex build chains. They MUST be distributed as binaries to be ready-to-use.
- **advanced**
- pip can actually install from any archive, wheel, or git/svn repo...
- ...that can be located on disk, or on a HTTP URL, or a personal pypi server.
- `pip install git+https://github.com/psf/requests.git@v2.25.0` for example (it can be useful for testing patches on a branch).
- `pip install https://download.pytorch.org/whl/cpu/torch-1.9.0%2Bcpu-cp39-cp39-linux_x86_64.whl` (that wheel is Python 3.9 on Linux).

- when installing from source, pip will automatically build the package. (it's not always possible, try building TensorFlow without the google build system :D)
- binary wheels can be python-version specific and OS specific, see manylinux specification to maximize portability.

conda

- **You are NOT permitted to use Anaconda or packages from Anaconda repositories for commercial use, unless you acquire a license.**
- Conda is a third party package manager from conda.
- It's popularized by anaconda, a Python distribution including most common data science libraries ready-to-use.
- You will use conda when you use anaconda.
- Packages are downloaded from the anaconda repo.
- It only installs precompiled packages.
- Conda has its own format of packages. It doesn't use wheels.
- `conda install` to install a package.
- `conda build` to build a package.
- conda can build the python interpreter (and other C packages it depends on). That's how an interpreter is built and bundled for anaconda.
- conda allows to install and upgrade the Python interpreter (pip does not).
- **advanced**
- Historically, the selling point of conda was to support building and installing binary packages, because pip did not support binary packages very well (until wheels and manylinux2010 spec).
- Emphasis on building packages. Conda has extensive build settings and it stores extensive metadata, to work with dependencies and build chains.
- Some projects use conda to initiate complex build systems and generate a wheel, that is published to pypi.org for pip.

easy_install/egg

- **For historical reference only. DO NOT USE**
- egg is an abandoned format of package, it was used up to mid 2010s and completely replaced by wheels.
- an egg is a zip archive, it contains python source files and/or compiled libraries.
- eggs are used with `easy_install` and the first releases of pip.
- `easy_install` was yet another package manager, that preceded pip and conda. It was removed in setuptools v58.3 (year 2021).
- it too caused a lot of confusion, just like pip vs conda :D
- egg files are slow to load, poorly specified, and OS specific.
- Each egg was setup in a separate directory, an `import mypackage` would have to look for `mypackage.py` in potentially hundreds of directories (how many libraries were installed?). That was slow and not friendly to the filesystem cache.

Historically, the above three tools were open-source and written in Python. However the company behind conda updated their Terms of Service in 2020 to prohibit commercial usage, watch out!

Funfact: The only strictly-required dependency to build the Python interpreter is zlib (a zip library), because compression is necessary to load more packages. Eggs and wheels packages are zip files.

Why so many options?

A good question.

Let's delve into the history of Python and computers. =D

Pure python packages have always worked fine with any of these packagers. The troubles were with not-only-Python packages.

Most of the code in the world depends on C. That is true for the Python interpreter, that is written in C. That is true for numerous Python packages, that are python wrappers around C libraries or projects mixing python/C/C++ code.

Anything that involves SSL, compression, GUI (X11 and Windows subsystems), math libraries, GPU, CUDA, etc... is typically coupled with some C code.

This creates troubles to package and distribute Python libraries because it's not just Python code that can run anywhere. The library must be compiled, compilation requires compilers and system libraries and third party libraries, then once compiled, the generated binary code only works for the specific system and python version it was compiled on.

Originally, python could distribute pure-python libraries just fine, but there was little support for distributing binary libraries. In and around 2010 you'd get a lot of errors trying to use `numpy` or `cassandra`. It downloaded the source and failed to compile, because of missing dependencies. Or it downloaded a prebuilt package (maybe an egg at the time) and it crashed with a SEGFAULT when used, because it was built for another system. It was a nightmare.

This was resolved by pip and wheels from 2012 onward. Then wait many years for people to adopt the tools and for the tools to propagate to stable Linux distributions (many developers rely on `/usr/bin/python`). The issues with binary packages extended to the late 2010s.

For reference, that's why the first command to run is `python3 -m venv myvenv && source myvenv/bin/activate && pip install --upgrade pip setuptools` on antiquated systems, because the OS comes with an old python+pip from 5 years ago that's buggy and can't recognize the current package format.

Conda worked on their own solution in parallel. Anaconda was specifically meant to make data science libraries easy to use out-of-the-box (data science = C and C++ everywhere), hence they had to come up with a package manager specifically meant to address building and distributing binary packages, conda.

If you install any package with `pip install xxx` nowadays, it just works. That's the recommended way to install packages and it's built-in in current versions of Python.

Share Improve this answer Follow

edited Nov 16, 2021 at 9:48

answered Aug 23, 2021 at 18:41

user5994461
5,607 3 36 58

35

Not to confuse you further, but you can also use pip within your conda environment, which validates the general vs. python specific managers comments above.

```
conda install -n testenv pip
source activate testenv
pip <pip command>
```

you can also add pip to default packages of any environment so it is present each time so you don't have to follow the above snippet.

Share Improve this answer Follow

answered May 13, 2015 at 14:25

vijay venkatesh
351 3 2

I thought this was not recommended? – endolith Feb 20, 2016 at 22:42

15 It's fully recommended to use pip inside of conda. It's better to install using conda, but for any packages that don't have a conda build, it's perfectly acceptable to use pip. – Bradley Kreider May 26, 2016 at 17:08

7 nit: seems like the phrase would be `fully supported` ? `fully recommended` implies, better to use pip than conda, within a conda enviornment, to my mind,a nd I'm not sure that is what you/they mean? – Hugh Perkins Jun 29, 2017 at 9:59

1 @HughPerkins what this means is that you should install pip inside your conda environment and use pip in that conda environment (only) when conda-formatted packages are not available from the major conda repos (like conda-forge or anaconda.org). Use pip install LAST to maintain the environment integrity, since pip does not do environment package integrity checking after it (greedily) installs packages. For more information see: docs.conda.io/projects/conda/en/latest/user-guide/tasks/... – Rich Lysakowski PhD Aug 5, 2022 at 4:45

I switched to pyenv in the end. After several years of dealing with eternal pip vs conda conflicts it was sooo relaxing to just use pip for everything, and everything just works. (Like, pyenv handles the ability to have multiple python versions on your system easily, which virtualenv doesn't directly handle for you; but also lets you simply do `pip install -r requirements.txt` , and similar) – Hugh Perkins Aug 5, 2022 at 10:35

33

Quote from Conda for Data Science article onto Continuum's website:

Conda vs pip

Python programmers are probably familiar with pip to download packages from PyPI and manage their requirements. Although, both conda and pip are package managers, they are very different:

- Pip is specific for Python packages and conda is language-agnostic, which means we can use conda to manage packages from any language Pip compiles from source and conda installs binaries, removing the burden of compilation
- Conda creates language-agnostic environments natively whereas pip relies on virtualenv to manage only Python environments Though it is recommended to always use conda packages, conda also includes pip, so you don't have to choose between the two. For example, to install a python package that does not have a conda package, but is available through pip, just run, for example:

```
conda install pip
pip install gensim
```

Share Improve this answer Follow

edited Feb 13, 2020 at 11:49

answered Jan 19, 2017 at 16:54

CheTesta
571 5 19

2 This was what I needed, previously i have installed the package in pip but I am unable to import it in my conda environment. This helped to solve the issue. – Goh Jia Yi Dec 19, 2021 at 15:07

1 You should install pip inside your conda environment and use pip in that conda environment (only) when conda-formatted packages are not available from the major conda repos (like conda-forge or anaconda.org). Use `pip install your_package` LAST to maintain the environment integrity, because pip does not do environment package integrity checking after it (greedily) installs packages. For more information see: docs.conda.io/projects/conda/en/latest/user-guide/tasks/... – Rich Lysakowski PhD Aug 5, 2022 at 4:46

25

`pip` is a package manager.

`conda` is both a package manager and an environment manager.

Detail:

	conda	pip
install python package	✅	✅
create virtual environment	✅, built-in	❌, requires `virtualenv` or `venv`
package format	`.tar.bz2` , `.conda`	`.whl` , `.tar.gz`
manages	binaries	wheel or source
can require compilers	❌	✅
package types	any	Python-only
dependency checks	✅	❌
package sources	Anaconda repo and Anaconda cloud	PyPI

Dependency check

> Pip and conda also differ in how dependency relationships within an environment are fulfilled. When installing packages, pip installs dependencies in a recursive, serial loop. No effort is made to ensure that the dependencies of all packages are fulfilled simultaneously. This can lead to environments that are broken in subtle ways, if packages installed earlier in the order have incompatible dependency versions relative to packages installed later in the order. In contrast, conda uses a satisfiability (SAT) solver to verify that all requirements of all packages installed in an environment are met. This check can take extra time but helps prevent the creation of broken environments. As long as package metadata about dependencies is correct, conda will predictably produce working environments.

References

- Understanding Conda and Pip

Share Improve this answer Follow

edited Mar 15, 2022 at 1:24

answered Sep 28, 2019 at 16:09
Simba
24.1k 7 64 76

7 I wonder what exactly is this "dependency check", pip is checking dependencies compatibility. `ERROR: package-a 0.5.9 has requirement package-b~=0.2.0, but you'll have package-b 1.0.1 which is incompatible.` (much faster than conda). – Pierre.Sassoulas Jul 23, 2020 at 13:52

The table is helpful, but the "can require compilers" line is a bit ambiguous. Conda can install a package that itself requires a compiler; the various flavors of the Stan probabilistic computing language are examples (e.g., PyStan). In this case, Conda will install a platform-specific compiler if necessary, e.g., llvm for macOS. – Tom Loredo Oct 13, 2022 at 20:43

"This check can take extra time..." I've literally waited over a weekend for an environment to be 'solved' (but ultimately wasn't). The solver doesn't scale well with number of dependencies: github.com/conda/conda/issues/11919 – EricRobertBrewer Sep 1 at 21:20

23

Quoting from Conda: Myths and Misconceptions (a comprehensive description):

...

Myth #3: Conda and pip are direct competitors

Reality: Conda and pip serve different purposes, and only directly compete in a small subset of tasks: namely installing Python packages in isolated environments.

Pip, which stands for **P**ip **I**nstalls **P**ackages, is Python's officially-sanctioned package manager, and is most commonly used to install packages published on the Python Package Index (PyPI). Both pip and PyPI are governed and supported by the Python Packaging Authority (PyPA).

In short, pip is a general-purpose manager for Python packages; conda is a language-agnostic cross-platform environment manager. For the user, the most salient distinction is probably this: pip installs python packages within any environment; conda installs any package within conda environments. If all you are doing is installing Python packages within an isolated environment, conda and pip+virtualenv are mostly interchangeable, modulo some difference in dependency handling and package availability. By isolated environment I mean a conda-env or virtualenv, in which you can install packages without modifying your system Python installation.

Even setting aside Myth #2, if we focus on just installation of Python packages, conda and pip serve different audiences and different purposes. If you want to, say, manage Python packages within an existing system Python installation, conda can't help you: by design, it can only install packages within conda environments. If you want to, say, work with the many Python packages which rely on external dependencies (NumPy, SciPy, and Matplotlib are common examples), while tracking those dependencies in a meaningful way, pip can't help you: by design, it manages Python packages and only Python packages.

Conda and pip are not competitors, but rather tools focused on different groups of users and patterns of use.

Share Improve this answer Follow

answered Apr 24, 2017 at 23:44
sancho.s
ReinstateMonicaCellio
14.8k 20 94 187

3 I'm not sure this is really true, beyond a market positioning perspective. For example, look at pytorch, which offers three types of install: conda, pip, source, pytorch.org , and recommends: conda – Hugh Perkins Jun 29, 2017 at 10:00

2 "Installing Python packages in isolated environments" is kind of the main thing most Python developers use pip for. – Nick Sep 20, 2017 at 19:56

@Nick isn't it when the developer is already in 'that' virtual environment? I think pip works in virtual environment and installs package as if it's installing for a system.But as sanchos.s said, it installs only python packages and doesn't take care for the underlying libraries. anyone please correct me if I'm wrong. – Chan Kim Jan 20, 2018 at 12:26

13

For WINDOWS users

"standard" packaging tools situation is improving recently:

- on pypi itself, there are now 48% of wheel packages as of sept. 11th 2015 (up from 38% in may 2015 , 24% in sept. 2014),
- the wheel format is now supported out-of-the-box per latest python 2.7.9,

"standard"+"tweaks" packaging tools situation is improving also:

- you can find nearly all scientific packages on wheel format at http://www.lfd.uci.edu/~gohlke/pythonlibs,
- the mingwpy project may bring one day a 'compilation' package to windows users, allowing to install everything from source when needed.

"Conda" packaging remains better for the market it serves, and highlights areas where the "standard" **should** improve.

(also, the dependency specification multiple-effort, in standard wheel system and in conda system, or buildout, is not very pythonic, it would be nice if all these packaging 'core' techniques could converge, via a sort of PEP)

Share Improve this answer Follow

edited Jun 21, 2016 at 16:49
Wilfred Hughes
30k 15 141 193

answered May 14, 2015 at 9:03
stonebig
1,193 1 9 13

9

(May 2023 UPDATE) This answer was derived from the one above by @user5994461

You can use pip for package management. Pip is the official built-in package manager for Python.org since Python 3.

pip is not a virtual environment manager.

pip

- **basics**
- pip is the default package manager for python
- pip is built-in as of Python 3.0
- Usage: `python3 -m venv myenv; source myenv/bin/activate; python3 -m pip install requests`
- Packages are downloaded from pypi.org, the official public python repository
- It can install precompiled binaries (wheels) when available, or source (tar/zip archive).
- Compiled binaries are important because many packages are mixed Python/C/other with third-party dependencies and complex build chains. They MUST be distributed as binaries to be ready-to-use.
- **advanced**
- pip can actually install from any archive, wheel, or git/svn repo...
- ...that can be located on disk, or on a HTTP URL, or a personal pypi server.
- `pip install git+https://github.com/psf/requests.git@v2.25.0` for example (it can be useful for testing patches on a branch).
- `pip install https://download.pytorch.org/whl/cpu/torch-1.9.0%2Bcpu-cp39-cp39-linux_x86_64.whl` (that wheel is Python 3.9 on Linux).
- when installing from source, pip will automatically build the package. (it's not always possible, try building TensorFlow without the google build system :D)
- binary wheels can be python-version specific and OS specific, see manylinux specification to maximize portability.

conda

- conda is an open source environment manager AND package manager maintained by the open source community. It is separate from Anaconda, Inc. and does not require a commercial license to use in any business environment.
- conda is also bundled into Anaconda Navigator, a popular commercial Python distribution from Anaconda, Inc. Anaconda) that includes most common data science and Python developer libraries ready-to-use.
- You will use conda when you use Anaconda Navigator GUI.
- Packages may be downloaded from conda-forge, anaconda repo4, and other public and private conda package "channels" (aka repos).
- It only installs pre-compiled packages.
- conda has its own package format. It doesn't use wheels.
- `conda install` will install a package.
- `conda build` will build a package.
- conda can build the Python interpreter (and other C packages it depends on). That's how an interpreter is built and bundled for Anaconda Navigator.
- conda allows users to install and upgrade the Python interpreter (pip does not).
- **advanced**

- Historically, one selling point of conda was to support building and installing binary packages, because pip did not support binary packages very well (until wheels and manylinux2010 spec).
- Emphasis on building packages. conda has extensive build settings and it stores extensive metadata, to work with dependencies and build chains.
- Some projects use conda to initiate complex build systems and generate a wheel, that is published to pypi.org for pip.
- conda emphasizes building and managing virtual environments. conda is by design a programming language-agnostic virtual environment manager. conda can install and manage other package managers such as npm, pip, and other language package managers.
- Can I use Anaconda Navigator packages for commercial use? The new language states that use by individual hobbyists, students, universities, non-profit organizations, or businesses with less than 200 employees is allowed, and all other usage is considered commercial and thus requires a business relationship with Anaconda. (as of Oct 28, 2020)
- **IF you are a large developer organization, i.e., greater than 200 employees, you are NOT permitted to use Anaconda or packages from Anaconda repository for commercial use, unless you acquire a license.**
- Pulling and using (properly open-sourced) packages from conda-forge repository do not require commercial licenses from Anaconda, Inc. Developers are free to build their own conda packages using the packaging tools provided in the conda-forge infrastructure.

easy_install/egg

- **For historical reference only. DO NOT USE**
- egg is an abandoned format of package, it was used up to mid 2010s and completely replaced by wheels.
- an egg is a zip archive, it contains python source files and/or compiled libraries.
- eggs are used with `easy_install` and the first releases of pip.
- `easy_install` was yet another package manager, that preceded pip and conda. It was removed in setuptools v58.3 (year 2021).
- it too caused a lot of confusion, just like pip vs conda :D
- egg files are slow to load, poorly specified, and OS specific.
- Each egg was setup in a separate directory, an `import mypackage` would have to look for `mypackage.py` in potentially hundreds of directories (how many libraries were installed?). That was slow and not friendly to the filesystem cache.

Funfact: The only strictly-required dependency to build the Python interpreter is zlib (a zip library), because compression is necessary to load more packages. Eggs and wheels packages are zip files.

Why so many options?

A good question.

Let's delve into the history of Python and computers. =D

Pure python packages have always worked fine with any of these packagers. The troubles were with not-only-Python packages.

Most of the code in the world depends on C. That is true for the Python interpreter, that is written in C. That is true for numerous Python packages, that are python wrappers around C libraries or projects mixing python/C/C++ code.

Anything that involves SSL, compression, GUI (X11 and Windows subsystems), math libraries, GPU, CUDA, etc... is typically coupled with some C code.

This creates troubles to package and distribute Python libraries because it's not just Python code that can run anywhere. The library must be compiled, compilation requires compilers and system libraries and third party libraries, then once compiled,

the generated binary code only works for the specific system and python version it was compiled on.

Originally, python could distribute pure-python libraries just fine, but there was little support for distributing binary libraries. In and around 2010 you'd get a lot of errors trying to use `numpy` or `cassandra`. It downloaded the source and failed to compile, because of missing dependencies. Or it downloaded a prebuilt package (maybe an egg at the time) and it crashed with a SEGFAULT when used, because it was built for another system. It was a nightmare.

This was resolved by pip and wheels from 2012 onward. Then wait many years for people to adopt the tools and for the tools to propagate to stable Linux distributions (many developers rely on `/usr/bin/python`). The issues with binary packages extended to the late 2010s.

For reference, that's why the first command to run is `python3 -m venv myvenv && source myvenv/bin/activate && pip install --upgrade pip setuptools` on antiquated systems, because the OS comes with an old python+pip from 5 years ago that's buggy and can't recognize the current package format.

Continuum Analytics (later renamed Anaconda, Inc.) worked on their own solution (released as Anaconda Navigator) in parallel. Anaconda Navigator was specifically meant to make data science libraries easy to use out-of-the-box (data science = C and C++ everywhere), hence they came up with a package manager specifically meant to address building and distributing binary packages, and built the environment manager -- conda -- into their distribution. It was later open-sourced and embraced by professional software engineers who frequently work with mixed-language environments.

If you install any package with `pip install xxx` nowadays, it USUALLY just works, until you try to use a package without binaries available for your platform. pip is built into current versions of Python, and it is a recommended way to install Python-only packages.

For mixed-language environments, conda is the way to go, because it will install pre-existing binary packages, regardless of their coding language, and use other package managers from other languages (npm, bower, to install their respective language packages.

Share Improve this answer Follow

edited May 25 at 6:08

answered Aug 5, 2022 at 5:31

Rich Lysakowski PhD
2,779 1 32 44

5

To answer the original question,
For installing packages, PIP and Conda are different ways to accomplish the same thing. Both are standard applications to install packages. The main difference is the source of the package files.

- PIP/PyPI will have more "experimental" packages, or newer, less common, versions of packages
- Conda will typically have more well established packages or versions

An important cautionary side note: If you use both sources (pip and conda) to install packages in the same environment, this may cause issues later.

- Recreate the environment will be more difficult
- Fix package incompatibilities becomes more complicated

Best practice is to select one application, PIP or Conda, to install packages, and use that application to install any packages you need. However, there are many exceptions or reasons to still use pip from within a conda environment, and vice versa. For example:

- When there are packages you need that only exist on one, and the other doesn't have them.
- You need a certain version that is only available in one environment

Share Improve this answer Follow

edited Jul 13, 2020 at 6:08

answered Jun 8, 2020 at 8:18

Donald S
1,603 1 10 26

4

> Can I use pip to install iPython?

Sure, both (first approach on page)

```
pip install ipython
```

and (third approach, second is `conda`)

> You can manually download IPython from GitHub or PyPI. To install one of these versions, unpack it and run the following from the top-level source directory using the Terminal:
>
> ```
> pip install .
> ```

are [officially recommended ways to install](#).

> Why should I use conda as another python package manager when I already have pip?

As said [here](#):

> If you need a specific package, maybe only for one project, or if you need to share the project with someone else, conda seems more appropriate.

Conda surpasses pip in ([YMMV](#))

- projects that use non-python tools
- sharing with colleagues
- switching between versions
- switching between projects with different library versions

> What is the difference between pip and conda?

That is extensively answered by everyone else.

Share Improve this answer Follow

answered Jul 25, 2017 at 15:33
serv-inc
36.1k 9 169 190

0

`pip` is for Python only

`conda` is only for Anaconda + other scientific packages like R dependencies etc. NOT everyone needs Anaconda that already comes with Python. Anaconda is mostly for those who do Machine learning/deep learning etc. Casual Python dev won't run Anaconda on his laptop.

Share Improve this answer Follow

answered May 3, 2018 at 1:21

Galapagos
669 6 13

1 simple explaination, but I was taught to go directly to Anaconda's website and download the Python 2.x or 3.x distribution. Why? > because it contains all packages that a student will need. Numpy, Scipy, matpliotlib, sklearn etc. This is exactly why there is a gap in understanding the finer fundamental details. Student – Rene Duchamp Sep 6, 2018 at 20:03

1 I know this is an old answer at this point, but what you've written about Conda/Anaconda seems completely false to me. – AMC Jan 9, 2020 at 1:10

@Galapagos Your answer is misleading and wrong. Anaconda is one of several industry standard Python distributions. conda is separate from Anaconda, but it used by Anaconda. Also today "Anaconda is mostly for those who do Machine learning/deep learning etc. Casual Python dev won't run Anaconda on his laptop." is blatantly wrong and outdated. Anaconda is the default for most kinds of analytics and data science work using Python today. – Rich Lysakowski PhD Sep 11, 2020 at 0:50

0

I may have found one further difference of a minor nature. I have my python environments under `/usr` rather than `/home` or whatever. In order to install to it, I would have to use `sudo install pip`. For me, the undesired side effect of `sudo install pip` was slightly different than what are widely reported elsewhere: after doing so, I had to run `python` with `sudo` in order to import any of the `sudo`-installed packages. I gave up on that and eventually found I could use `sudo conda` to install packages to an environment under `/usr` which then imported normally without needing `sudo` permission for `python`. I even used `sudo conda` to fix a broken `pip` rather than using `sudo pip uninstall pip` or `sudo pip --upgrade install pip`.

Share Improve this answer Follow

edited Jan 9, 2019 at 12:12
chb
1,747 7 26 48

answered Jan 9, 2019 at 11:48
J B's
J B
360 1 6